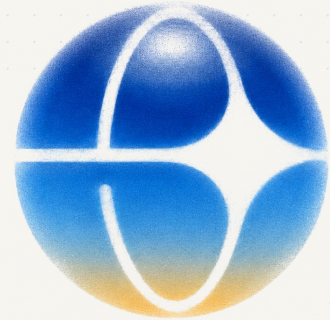




VERIFIED TESTNET FACTS + PROPOSED OPENARC ARCHITECTURE

Marketing + *technical brief.*



A combined product narrative, verified Arc Testnet reference, system architecture, evidence model, release plan, and communications system.

POSITIONING

Agent activity, made legible

VERIFIED TESTNET

Current chain, contracts, and limits

SYSTEM DESIGN

Private evidence graph + narrow API

BUILD + LAUNCH

Schemas, tests, migration, and claims

THE NON-NEGOTIABLE LINE

Observe and reconcile. Never pretend to know the missing evidence.

Arc Testnet concept. No custody, signing, hidden reasoning, or mainnet claim.

BRIEF MAP

38 sections of verified Testnet context, product architecture, evidence, positioning, and launch material.

The Markdown package is editable. This PDF is the visually verified distribution copy.

01	1. How to use this document	20	20. Positioning hierarchy
02	2. Mandatory public-launch fields	21	21. Message pillars
03	3. Executive product truth	22	22. Audience map
04	4. Product story	23	23. Brand voice
05	5. Proposed initial product	24	24. Tagline and headline bank
06	6. What OpenArc is not	25	25. Landing-page narrative
07	7. The evidence model	26	26. Claims matrix
08	8. Verified Arc Testnet specification	27	27. Product comparison language
09	9. Arc-specific implementation rules	28	28. Launch campaign architecture
10	10. Arc agent identity and job evidence	29	29. Suggested 14-post sequence
11	11. x402 and Gateway evidence flow	30	30. Social post bank
12	12. Proposed system architecture	31	31. Thread scripts
13	13. Connector and application boundaries	32	32. FAQ and reply bank
14	14. Evidence schema and correlation	33	33. Content production guide
15	15. Local encryption and recovery	34	34. CTA library
16	16. Privacy and security controls	35	35. Measurement
17	17. Failure and degraded states	36	36. Incident communication templates
18	18. Build plan and release gates	37	37. Publication checklist
19	19. Arc mainnet migration gate	38	38. Source registry

SECTION 01

1. How to use this document

Treat this sourcebook as the marketing source of truth until the product has a versioned implementation and release record. Do not fill missing facts with plausible copy.

Every draft must preserve these distinctions:

- Arc is the network; OpenArc is an independent proposed application.
- Circle provides infrastructure including USDC, Agent Stack, and Gateway; OpenArc is not Circle software and is not endorsed unless a written record says otherwise.
- OpenArc is being built first for Arc Testnet. Arc has announced public mainnet for September 16, 2026, but OpenArc mainnet support is not a present capability and the public mainnet configuration is not yet in Arc's network references.
- An onchain transaction proves execution or settlement, not private intent, fulfillment quality, or model reasoning.
- A local OpenArc policy describes the owner's intended boundary. It is not enforced by another wallet unless that wallet returns explicit enforcement evidence.
- OpenArc observes and reconciles. It does not take custody, collect seed phrases, sign, or broadcast transactions.
- Product plans, testnet demonstrations, and shipped production capabilities must never be blended.

For each requested marketing asset, provide:

1 Channel and format.

2 Audience and objective.

3 Final copy.

4 Suggested visual or product proof.

5 CTA.

6 Claims used and supporting source.

7 Missing fields or approvals.

8 A final check for implied endorsement, mainnet availability, invisible model reasoning, absolute privacy, guaranteed enforcement, or invented usage data.

SECTION 02

2. Mandatory public-launch fields

Do not infer or publish these fields until confirmed:

```
PRODUCT_URL=[CONFIRM BEFORE POSTING]
OFFICIAL_X_HANDLE=[CONFIRM BEFORE POSTING]
OFFICIAL_COMMUNITY_URL=[CONFIRM BEFORE POSTING]
OFFICIAL_GITHUB=[CONFIRM BEFORE POSTING]
LEGAL_OPERATOR=[CONFIRM BEFORE POSTING]
JURISDICTION=[CONFIRM BEFORE POSTING]
SUPPORT_CONTACT=[CONFIRM BEFORE POSTING]
SECURITY_CONTACT=[CONFIRM BEFORE POSTING]
ARC_TESTNET_DEPLOYED_VERSION=[CONFIRM AFTER DEPLOYMENT]
ARC_TESTNET_DEPLOYED_SHA=[CONFIRM AFTER DEPLOYMENT]
ARC_MAINNET_STATUS=[DO NOT CLAIM UNTIL OFFICIAL PARAMETERS AND SUPPORT EXIST]
SUPPORTED_AGENT_CONNECTORS=[CONFIRM FROM DEPLOYED BUILD]
SUPPORTED_PAYMENT_PROTOCOLS=[CONFIRM FROM DEPLOYED BUILD]
SUPPORTED_EVIDENCE_STATES=[CONFIRM FROM DEPLOYED BUILD]
DATA_RETENTION=[CONFIRM FROM IMPLEMENTED PRIVACY MODEL]
SECURITY_REVIEW_STATUS=[CONFIRM BEFORE PUBLIC FUNDS]
TRADEMARK_REVIEW_STATUS=[CONFIRM BEFORE PUBLIC BRAND LAUNCH]
PRICING=[CONFIRM BEFORE POSTING]
```

No token, points system, governance right, revenue share, or access requirement is part of this sourcebook.

SECTION 03

3. Executive product truth

One sentence

OpenArc brings an agent's permissions, actions, payments, receipts, and settlements into one evidence trail.

Short description

OpenArc is a proposed private intelligence and investigation layer for economic agents on Arc. It helps operators understand what an agent was allowed to do, what it attempted, what it paid, what a service returned, and what ultimately settled - without turning OpenArc into the wallet or transaction executor.

Full promise

Follow the complete arc of an economic action. Connect owner-supplied policies, signed mandates, agent events, x402 payment evidence, service receipts, Gateway records, and Arc transactions. OpenArc shows where those records agree, where they conflict, and what cannot be verified.

Core hook

See the full arc of every agent action.

Plain-English hook

See what was allowed. See what happened.

Developer hook

One evidence model for intent, execution, fulfillment, and settlement.

The problem

When software can spend money, a transaction hash is not enough. Operators may need several systems to answer basic questions:

- Who authorized the action?
- Which limit or merchant rule applied?
- What did the agent request?
- What amount did it sign?
- Did the API or merchant respond?
- Did payment actually settle?
- Did the action match its original permission?

Wallets, agent logs, HTTP payment headers, facilitators, and block explorers each show one part. None necessarily presents the complete human-readable trail.

The product answer

OpenArc normalizes those records into one action envelope. It preserves the difference between local intent, signed evidence, agent-reported events, provider claims, facilitator state, and onchain facts. It explains the action without fabricating the missing links.



SECTION 04

4. Product story

Arc moves value

Arc and Circle infrastructure are designed to give applications and agents programmable stablecoin rails, predictable settlement, and agent-oriented payment tools.

Agents create a new observability problem

An agent may discover a service, interpret a price, produce a payment authorization, receive a result, and trigger settlement faster than a human can inspect each step. Traditional wallet history begins too late in that story.

OpenArc makes the path legible

OpenArc joins the supplied evidence into a chronological and causal view. The human remains responsible for defining policy, approving integrations, and deciding what to trust.

The defining line

Arc is the settlement environment. OpenArc is the evidence and investigation interface above it.

SECTION 05

5. Proposed initial product

Use future or testnet language until the exact build is deployed.

Agent activity

- Read-only Arc Testnet wallet and USDC activity.
- Action timelines across request, authorization, fulfillment, and settlement.
- Exact timestamps, values, destinations, and source classes.
- Explicit incomplete, stale, conflicting, and unsupported states.

Policy workspace

- Locally encrypted spending limits and service allowlists.
- Human approval requirements and expirations.
- Comparison of observed action evidence against the applicable local policy.
- No enforcement claim unless a connected wallet supplies proof of enforcement.

Payment investigation

- Bounded x402 payment requirements and authorization metadata.
- Provider response and fulfillment receipt when supplied.
- Gateway verification/batching state where supported.
- Arc transaction and settlement reference.
- Detection of mismatched amount, asset, network, recipient, expiry, or action ID.

Source center

- Exact destination and capability of each integration.
- Freshness, outage, and degraded-state labels.
- Privacy receipts for approved fields.
- Clear separation between provider statements and independently observed facts.

Encrypted workspace

- Local agent names, notes, policies, and evidence associations.
- Lock, encrypted backup, recovery, and deletion.
- No seed phrase or private-key collection.



SECTION 06

6. What OpenArc is not

- Not an agent wallet.
- Not a blockchain or explorer replacement.
- Not a payment facilitator.
- Not a custody or key-management product.
- Not an autonomous trader.
- Not an AI model that claims to know hidden reasoning.
- Not a universal agent identity or reputation registry.
- Not a compliance certificate.
- Not a guarantee of provider fulfillment or settlement.
- Not an Arc or Circle product.



SECTION 07

7. The evidence model

OpenArc should describe records using six evidence classes:

- 1 Local declaration:** a label, policy, or note stored by the owner.
- 2 Signed authorization:** a mandate or payment authorization with validated signature and scope.
- 3 Agent-reported event:** an event emitted by an agent or tool connector.
- 4 Provider receipt:** a service's bounded claim that it returned a resource or completed a step.
- 5 Facilitator record:** a Gateway or payment-service verification/settlement state.
- 6 Onchain fact:** a transaction or state read from the configured network.

Approved explanation:

OpenArc does not flatten every record into “verified.” It shows which system is authoritative for each part of the action.



SECTION 08

8. Verified Arc Testnet specification

These are current official Testnet facts reviewed on 2026-09-01. They are not Arc mainnet values and must be rechecked at every release.

- EVM chain ID: [5042002](#).
- CAIP-2 network: [eip155:5042002](#).
- Primary HTTP RPC: <https://rpc.testnet.arc.io>.
- Primary WebSocket RPC: <wss://rpc.testnet.arc.io>.
- Explorer: <https://testnet.arcscan.app>.
- Faucet: <https://faucet.circle.com>; Testnet assets have no real value.
- Consensus: Malachite BFT with a permissioned validator set.
- Current observed block time: approximately [0.48 s](#) on Testnet.
- Finality: deterministic when committed; current integration rule is one block.
- Execution: EVM; the current Arc differences reference names the Osaka baseline.
- Gas asset: USDC, never ETH.
- USDC ERC-20 interface: [0x3600](#).
- Native USDC uses 18-decimal accounting; the ERC-20 interface uses 6 decimals.
- Gateway domain: [26](#); this is not the EVM chain ID.
- GatewayWallet: [0x0077777d7EBA4688BDeF3E311b846F25870A19B9](#).
- GatewayMinter: [0x002222ABE238Cc2C7Bb1f21003F0a260052475B](#).

OpenArc should pin these values in a versioned Testnet capability registry. A response from another chain, contract, or provider origin fails closed.

The brief must not present confidential transfers, selective disclosure, post-quantum wallet signatures, or Arc mainnet addresses as available Testnet features. Current Arc documentation describes those capabilities as roadmap or mainnet work.



SECTION 09

9. Arc-specific implementation rules

Arc is EVM-compatible, but a generic Ethereum indexer is not sufficient without Arc normalization.

- Native and ERC-20 USDC are two interfaces over one balance, not two assets.
- Never compare the 18-decimal native raw value with the 6-decimal ERC-20 raw value before exact conversion.
- The ERC-20 view truncates values below 1 micro-USDC; an ERC-20 **0** does not prove the native balance is exactly zero.
- Native USDC movements emit EIP-7708-style **Transfer** logs from a system emitter. OpenArc must identify the emitter and avoid double counting.
- Block timestamps are non-decreasing, not strictly increasing. Order evidence by block number, transaction index, and log index.
- **PREVRANDAO** returns zero and is not a randomness source.
- EIP-4844 blob transactions are not supported on the current network.
- USDC blocklist and Arc native-value rules can make a value call revert even when the payer has enough balance.
- Fees are exact USDC values. The currently documented Testnet base-fee floor is 20 Gwei in native accounting, but the UI must not turn that into a mainnet SLA.

OpenArc stores the exact block number, block hash, transaction hash, receipt status, emitter, and log index behind every onchain conclusion. Deterministic finality reduces confirmation waiting; it does not remove the need for provenance.



SECTION 10

10. Arc agent identity and job evidence

Arc's documented agentic surfaces make the product technically relevant beyond generic wallet monitoring.

ERC-8004 on Arc Testnet

- IdentityRegistry: [0x8004A818BFB912233c491871b3d84c89A494BD9e](#).
- ReputationRegistry: [0x8004B663056A597Dffe9eCcc1965A193B7388713](#).
- ValidationRegistry: [0x8004Cb1BF31DAf7788923b405b754f57acEB4272](#).

OpenArc can show ownership, metadata URI, external feedback, and validation responses. It must not turn a registry token into proof of real-world identity or flatten observer feedback into a universal trust score. Metadata remains application-defined and the source of every claim stays visible.

ERC-8183 reference job on Arc Testnet

Arc documents an AgenticCommerce reference implementation at:

[0x0747EEf0706327138c69792bF28Cd525089e4583](#)

Its lifecycle covers job creation, provider budget, USDC escrow funding, deliverable hash submission, evaluation, and completion or rejection. OpenArc can reconcile those onchain states with local policy and offchain fulfillment evidence. The contract is a Testnet reference implementation, not proof that all Arc agent jobs use it or that the address will exist on mainnet.



SECTION 11

11. x402 and Gateway evidence flow

x402 is the payment negotiation layer, not settlement itself.

```
request resource
  <- 402 + PAYMENT-REQUIRED
sign exact authorization
request + PAYMENT-SIGNATURE
  <- resource + PAYMENT-RESPONSE
Gateway: received -> batched -> confirmed -> completed | failed
Arc: exact transaction or contract evidence where applicable
```

Circle Gateway Nanopayments currently supports Arc Testnet as [arcTestnet](#), Gateway domain [26](#), and network [eip155:5042002](#). The documented method uses an EIP-3009 authorization under the [GatewayWalletBatched](#) EIP-712 domain.

OpenArc keeps normalized requirement, authorization, response, transfer, and settlement metadata separate. The default record stores a safe resource origin, local label, canonical digest, amount, recipient, asset, network, validity, nonce, and source times. It does not persist a reusable signature, API secret, private key, full paid response, or sensitive resource URL.

[received](#), [batched](#), and [confirmed](#) are intermediate Gateway states. [completed](#) is not silently substituted for provider fulfillment, and a provider receipt is not silently substituted for settlement.



SECTION 12

12. Proposed system architecture

OpenArc is browser-private first, with a narrow normalization service.

```
BROWSER
  encrypted IndexedDB Vault
  policy evaluator
  evidence graph + timeline
  private correlation engine
    |
    | exact consented public fields
    v
OPENARC API
  strict schemas + Origin boundary
  Arc RPC normalizer
  optional Circle/Gateway connector
  rate, subcall, size, time, and daily-budget limits
    |
    +---- Arc JSON-RPC / WebSocket
    +---- Circle Wallets / Gateway APIs
```

The browser owns agent labels, wallet associations, policy text, investigation notes, evidence relationships, and encrypted archives. It aborts in-flight work and invalidates late writes on lock, delete, import, or workspace replacement.

The API receives one bounded public identifier or evidence reference per explicit request. It pins chain, contract, and provider identity; returns only versioned allowlisted fields; sends **Cache-Control: no-store**; and logs route template, status, latency, and failure class rather than request values.

No server database is needed for the MVP evidence workspace. Redis may support HMAC-derived abuse limits, provider budgets, and short-lived locks, but may not hold raw wallet addresses, prompts, policies, or action associations.



SECTION 13

13. Connector and application boundaries

Each connector publishes a versioned capability manifest:

```
id + adapter version
testnet or mainnet environment
CAIP network
fact classes read
writes = false for MVP
authentication class
exact fields released
source retention
freshness rule
rate, size, subcall, and time limits
reviewed source revision
```

Proposed app-only API routes:

```
GET /v1/private/capabilities
POST /v1/private/arc/account-snapshot
POST /v1/private/arc/transaction-evidence
POST /v1/private/arc/agent-registry-evidence
POST /v1/private/arc/job-evidence
POST /v1/private/gateway/transfer-evidence
```

These routes are not a general public data API. Unknown fields fail validation. One wallet, transaction, agent ID, job ID, or transfer ID is accepted per call. Local labels, policies, prompts, response bodies, private URLs, wallet cookies, and unrelated history are not sent.

A Circle Agent Stack adapter may consume user-approved wallet identity and transaction evidence after privacy and commercial review. The current Circle CLI reference describes spending-policy inspection and setup as mainnet-only, so the Testnet MVP says [LOCAL MONITORING POLICY](#) unless a wallet returns enforcement proof.



SECTION 14

14. Evidence schema and correlation

The principal local record is a versioned action envelope:

```
ActionEnvelope
  actionId + agentId + actionType
  createdAt
  append-only state transitions
  policy evaluation + matched local rule IDs
  intent evidence IDs
  attempt evidence IDs
  payment evidence IDs
  fulfillment evidence IDs
  settlement evidence IDs
  reconciliation rule version
  gaps + conflicts
```

Every evidence record includes class, exact source, observation time, source event time, subject, digest, bounded normalized fields, and explicit limitations.

For x402, correlation requires more than matching amount and addresses:

```
network + asset + payer + recipient + exact base-unit amount
authorization nonce + validity window
safe resource origin + local digest
Gateway transfer ID when returned
exact Arc block / transaction / log tuple when available
```

OpenArc compares integer base units, never JavaScript floating point. Network, token, recipient, amount, nonce, validity, or action-ID conflicts produce **CONFLICTING_EVIDENCE**. Missing unique linkage remains an ambiguity instead of a best-guess match.



SECTION 15

15. Local encryption and recovery

The proposed Vault uses WebCrypto AES-256-GCM, a random data-encryption key, a versioned passphrase-derived wrapping key, a fresh 96-bit IV for every write, and authenticated additional data binding vault ID, record ID, schema, and key version. The unlocked session key is nonextractable.

Plain IndexedDB metadata contains only cryptographic bootstrap and version data. Names, addresses as local associations, policies, notes, record types, counts, timestamps, receipts, and evidence graphs remain encrypted.

Lock clears application state, revokes object URLs, aborts requests, and bumps a session generation. No async result can persist after that boundary. The product does not promise JavaScript heap erasure or protection from a compromised device, malicious extension, XSS, or hostile future same-origin code.

Encrypted export, import, recovery, and delete cover every evidence schema. An import cannot replace the current Vault until the complete archive is bounded, authenticated, decrypted, and validated in memory.



SECTION 16

16. Privacy and security controls

Never collect:

- Seed phrases, private keys, or wallet key shares.
- Circle entity secrets or OTP codes in the browser.
- Reusable payment signatures in persistent storage.
- Hidden model reasoning or chain-of-thought.
- Full private prompts or paid resource bodies by default.
- Unrelated wallet or browser history.

Before an optional network call, OpenArc saves an encrypted permission receipt with destination, exact released public fields, purpose, cookie behavior, provider retention, OpenArc no-store behavior, approval time, and outcome. If the receipt cannot be saved, nothing is sent. A failed provider still remains named.

The hosted API uses exact production HTTPS origin checks, strict body limits, provider-host allowlists, redirect rejection, response allowlists, timeouts, response-size caps, concurrency and subcall caps, atomic daily budgets, and route-template logging. Required budget infrastructure fails closed in production.



SECTION 17

17. Failure and degraded states

- Arc RPC outage: retain prior encrypted evidence, mark it stale, add no new conclusion.
- Wrong chain ID: reject the response and show configuration failure.
- Block-anchor mismatch: discard the whole multi-call result as source conflict.
- Gateway outage: preserve requirement and authorization; settlement remains unverified.
- Missing provider response: payment can settle while fulfillment remains unverified.
- Missing Arc receipt: provider/Gateway evidence remains; onchain settlement is unverified.
- Conflicting amount or recipient: show [CONFLICTING_EVIDENCE](#) and cite both sources.
- Unknown agent identity: show wallet or registry facts without calling it a verified agent.
- Local quota error: abort and say the new evidence was not saved.
- Vault lock mid-request: abort and discard; no late persistence.
- Rate or budget exhaustion: actionable 429/503 with prior evidence unchanged.
- Unsupported record version: fail closed while preserving unlock, export, and delete.

SECTION 18

18. Build plan and release gates

Build sequence

- 1 Local evidence types, state machine, exact arithmetic, fixtures, and graph.
- 2 Encrypted Vault with lock, export, import, recovery, delete, and cross-tab revision protection.
- 3 Arc chain registry, one-address snapshot, receipt/event normalization, and source-health UI.
- 4 Read-only ERC-8004 identity and ERC-8183 job evidence.
- 5 One controlled x402 resource and Gateway transfer reconciliation.
- 6 Signed generic agent-event connector.
- 7 Optional Circle Wallets or AP2 adapters only after separate privacy, signature, commercial, and threat-model review.

Verification gate

- Exact Arc constants and provider origins.
- Native/ ERC-20 conversion, truncation, EIP-7708 classification, and no double count.
- Strict x402 and Gateway schema, status, replay, expiry, and mismatch tests.
- Every evidence transition and conflict state.
- Private canaries absent from URLs, bodies, headers, logs, Redis, plaintext storage, and server persistence.
- Receipt-before-network, abort-on-lock, and no-late-write behavior.
- Complete encrypted export/delete/import/recovery journey.
- Chromium, WebKit, mobile, keyboard, screen-reader, and reduced-motion journeys.
- Exact deployed SHA plus live controlled Arc Testnet evidence and outage drills.

SECTION 19

19. Arc mainnet migration gate

Arc mainnet is a new configuration and review, not a Testnet feature toggle.

Required before a claim or connection:

- 1 Official mainnet chain ID, CAIP identifier, RPC, WebSocket, explorer, status, USDC interface, Gateway/CCTP domains and contracts, agent-registry/job deployments, and Wallets/Agent Stack capabilities.
- 2 Current finality, fee, privacy, signature, and EVM-difference documentation.
- 3 Provider terms, retention, authentication, rate limits, and hard cost ceilings.
- 4 Independent review of local encryption, API schemas, connector authentication, deployment, monitoring, backup, and rollback.
- 5 Operator identity, support/security contacts, privacy/terms, jurisdiction, and trademark/non-affiliation review.
- 6 Exact-SHA staging and production drills with a separate mainnet feature flag defaulting false.

Testnet and mainnet records must remain cryptographically and visually distinct. The first mainnet mode stays read-only and non-custodial.



SECTION 20

20. Positioning hierarchy

Category

Agent operations intelligence and investigation.

Primary differentiator

One evidence arc connects permission, attempt, payment, fulfillment, and settlement without pretending that any single source proves the whole story.

Supporting differentiators

- Human-readable without hiding identifiers and evidence.
- Private local context rather than a server-owned behavioral profile.
- Built around payment and settlement state, not chat transcripts.
- Protocol-aware but not dependent on one agent framework.
- Observability without custody or execution.

Positioning statement

For teams and individuals operating economic agents who need to understand more than a wallet balance, OpenArc is the private evidence and investigation layer that connects permissions, requests, payments, receipts, and settlement. Unlike a wallet, explorer, or agent log, OpenArc reconciles the whole supplied action path while keeping unknowns visible.

SECTION 21

21. Message pillars

Pillar 1: The whole action

Start before the payment and continue after it. The emotional benefit is context.

Proof language:

- Permission, attempt, authorization, fulfillment, and settlement appear as separate states.
- A complete timeline can show the source behind each transition.

Pillar 2: Human authority stays visible

Policies and signed mandates should remain connected to the actions they govern. The emotional benefit is control.

Proof language:

- Local budgets and allowlists are encrypted with the workspace.
- Missing or mismatched authorization is a first-class investigation state.

Pillar 3: Evidence without false certainty

The system names what each source proves and what it cannot prove. The emotional benefit is earned trust.

Proof language:

- Onchain settlement is not described as proof of private intent.
- A provider receipt is not described as independent proof of service quality.

Pillar 4: Private operating context

Agent names, policies, notes, and cross-source relationships should stay encrypted locally by default. The emotional benefit is ownership of operational context.

Pillar 5: Built for machine-speed money

Software needs payment rails; people need understandable oversight. The emotional benefit is readiness without surrendering judgment.



SECTION 22

22. Audience map

Agent builder

Question: "Why did this workflow spend money, and where did it fail?" Message: Trace the action across tool, payment, service, and settlement layers.

Agent operator or founder

Question: "Are my agents staying inside the boundaries I intended?" Message: Keep policy context beside every observed action and exception.

Finance or operations lead

Question: "What is authorized, pending, settled, or unreconciled?" Message: Replace disconnected exports with one evidence-state model.

Security or risk reviewer

Question: "Which source supports this conclusion?" Message: Inspect signed, provider, facilitator, and onchain evidence separately.

Arc builder

Question: "How do users understand agentic activity on Arc?" Message: Add a human-readable investigation surface without building a second wallet or explorer.

Curious user

Question: "What did the agent actually do with the money?" Message: See the action in plain English, along with what remains unknown.



SECTION 23

23. Brand voice

Use:

- Calm confidence.
- Short explanations anchored to visible evidence.
- Direct statements of uncertainty.
- Exact terms when they matter: authorization, recipient, expiry, settlement, fulfillment, source, timestamp.
- “Supplied,” “observed,” and “reconciled” instead of universal “verified.”

Avoid:

- Agent hype, sentient language, or claims that agents are replacing people.
- “The single source of truth” without qualification.
- “Trustless,” “fully autonomous,” or “compliance-ready” as broad adjectives.
- Claims that OpenArc understands hidden chain-of-thought.
- Generic AI words such as revolutionary, intelligent, next-generation, or game-changing without mechanism.
- Token, price, or investment language.
- Implied partnership with Arc, Circle, Google, Coinbase, or any framework.

Preferred vocabulary:

- Evidence, permission, action, authorization, fulfillment, settlement, reconcile, inspect, policy, source, exception, operator.

Words requiring context:

- **Verified:** Name the exact signature, provider, or onchain check.
- **Policy:** Say whether it is local monitoring or wallet-enforced.
- **Agent:** Say who classified the address or system as an agent.
- **Private:** State what is encrypted/local and name the threat-model limits.
- **Complete:** Name the action type and required evidence layers.
- **Real time:** Provide a timestamp and polling/streaming behavior.



SECTION 24

24. Tagline and headline bank

Primary candidates:

- See the full arc of every agent action.
- See what was allowed. See what happened.
- Agent activity, made legible.
- The evidence layer for autonomous finance.
- Follow the money. Keep the context.
- From intent to settlement, in one view.
- Your agents move fast. Your oversight should keep up.
- Understand every step between permission and payment.

Operator headlines:

- A transaction hash is the end of the story, not the beginning.
- Know what your agent attempted - not just what finally settled.
- Bring agent policy, payment, and settlement into one investigation.
- When software spends money, evidence matters.
- Inspect the action without surrendering the keys.

Developer headlines:

- One schema for intent, authorization, fulfillment, and settlement.
- Debug agent payments across HTTP, Gateway, and Arc.
- Stop reconciling agent actions across five different logs.
- Give operators a readable trail without rebuilding your wallet.

Arc-specific headlines:

- The control and evidence layer for agentic activity on Arc.
- Arc gives agents programmable money. OpenArc makes it understandable.
- Built to investigate machine-speed USDC activity.

Use the Arc-specific set only with a visible non-affiliation statement and an accurate Testnet/mainnet status.



SECTION 25

25. Landing-page narrative

Hero

Eyebrow:

AGENT OPERATIONS · ARC TESTNET CONCEPT

Headline:

See the full arc of every agent action.

Subheadline:

OpenArc connects permissions, requests, payments, receipts, and settlement so people can understand what an economic agent was allowed to do, what it attempted, and what actually happened.

Primary CTA before deployment:

Follow the Testnet build

Secondary CTA:

Read the evidence model

Boundary line:

Read-only by design. No custody. No seed phrases. No transaction signing.

Problem section

Headline:

A wallet shows the transfer. It does not show the whole decision.

Body:

Agent activity crosses policies, tools, APIs, payment protocols, facilitators, and blockchains. OpenArc is designed to reconcile those records without pretending they all prove the same thing.

Product section

Cards:

1 **Permission** - Which local policy or signed mandate applied?



- 2 **Attempt** - What did the connected agent or tool report doing?
- 3 **Payment** - What asset, amount, destination, and expiry were authorized?
- 4 **Fulfillment** - What did the service claim to return?
- 5 **Settlement** - What actually finalized through Gateway or Arc?
- 6 **Reconciliation** - Which fields agree, conflict, or remain unknown?

Privacy section

Headline:

The most sensitive context should not become another behavioral database.

Body:

OpenArc is designed to keep agent labels, local policies, investigation notes, and evidence relationships encrypted in the user's workspace. Optional network requests should disclose exactly which public fields leave the device.

Boundary section

Headline:

Oversight, not custody.

Body:

OpenArc does not hold keys, sign transactions, or claim to enforce another wallet's rules. Execution remains with the agent wallet and its approved infrastructure.

Final CTA

Help shape the first Arc Testnet investigation.

SECTION 26

26. Claims matrix

Claim: “Agent control layer”

Use before enforcement integrations: OpenArc provides local policy comparison, visibility, and investigation controls. Do not extend to: preventing another wallet from sending a transaction.

Safer early wording: **agent oversight and investigation layer**.

Claim: “Source of truth”

Use only with qualification: OpenArc presents a unified evidence view over multiple authoritative sources. Do not extend to: knowing hidden intent, service quality, or facts no source supplied.

Safer wording: **source-of-truth interface** or **reconciled evidence layer**.

Claim: “Private workspace”

Use after implementation: Named local records are encrypted before browser storage and are absent from ordinary server persistence. Do not extend to: compromised devices, browser extensions, XSS, unlocked access, or a malicious future deployment.

Claim: “Verified action”

Avoid as a standalone phrase. Name the check:

- Signature verified.
- Gateway verification received.
- Arc settlement observed.
- Provider receipt supplied.
- Policy fields matched.

Claim: “Arc support”

Before deployment: **Designed for an Arc Testnet prototype**. After exact testnet deployment: **Available for the documented Arc Testnet journey**. Do not say: Arc mainnet-ready, officially supported by Arc, or production-ready.

Claim: “Real time”

Use only when a deployed connector has a defined observation interval and visible timestamp. Safer wording: **current observed state** or **latest successful observation**.

Claim: “Policy enforcement”

Use only when the connected wallet confirms an enforced rule and the integration has been tested. Otherwise say: **local monitoring policy** or **policy simulation**.

Claim: “Complete audit trail”

Use only for a named action type when every required evidence record exists. Safer wording: **complete supplied evidence arc**.



SECTION 27

27. Product comparison language

OpenArc versus a wallet

A wallet holds or controls assets and submits actions. OpenArc is designed to explain and reconcile the evidence around those actions without holding keys.

OpenArc versus an explorer

An explorer shows public blockchain activity. OpenArc begins earlier - with policy, intent, tool events, and payment authorization - and continues through fulfillment and settlement.

OpenArc versus agent observability

Agent observability tools trace prompts, models, tokens, and tool calls. OpenArc focuses on economic authority, payment evidence, service receipts, and settlement.

OpenArc versus compliance software

Compliance systems evaluate regulatory and organizational requirements. OpenArc can display supplied policy and evidence, but it does not certify legal compliance.

OpenArc versus ChainScout

ChainScout is a private asset-intelligence dashboard for human-owned wallets, tokens, tokenized assets, and collectibles. OpenArc adapts the privacy and evidence principles to economic agents, focusing on permissions, machine-initiated actions, API payments, service receipts, and settlement. It is a distinct product concept, not a rename or automatic migration of ChainScout.

SECTION 28

28. Launch campaign architecture

Phase A: Establish the missing layer

Objective: Make the gap between a transaction and a complete agent action obvious.

Content:

- "A transaction hash cannot tell you why an agent acted."
- Diagrams of disconnected wallet, agent, service, and settlement records.
- Short explanations of authorized versus settled versus fulfilled.

Phase B: Introduce the evidence arc

Objective: Own a simple mental model.

Content:

- Intent -> permission -> attempt -> payment -> fulfillment -> settlement.
- One sample action with a missing receipt.
- One sample action with a policy mismatch.

Phase C: Show the Arc Testnet prototype

Objective: Demonstrate the product using controlled test funds and fictional labels.

Content:

- One labeled agent wallet.
- One x402 test resource.
- One bounded local policy.
- One reconciled evidence timeline.
- One outage or incomplete-evidence state.

Phase D: Invite builders

Objective: Validate connector and investigation needs.

Content:

- Publish the proposed generic agent-event schema.
- Ask builders which receipts and identifiers they can safely emit.
- Collect failure cases, not wallet keys or production secrets.

Phase E: Mainnet readiness - not an OpenArc launch countdown

Objective: Show disciplined preparation after official network parameters exist.

Content:

- Capability matrix.



- Security and privacy review.
- Exact mainnet source identity.
- Testnet-to-mainnet incompatibility checks.
- Explicit activation decision.

Arc has announced public mainnet for September 16, 2026. That date may be cited from Arc's announcement, but it must not be presented as an OpenArc availability date. OpenArc mainnet support remains gated on the configuration and review in section 19.



SECTION 29

29. Suggested 14-post sequence

- 1 A transaction hash is not an explanation.
- 2 The six evidence classes behind an economic agent action.
- 3 Authorized is not settled.
- 4 Settled is not fulfilled.
- 5 Why an agent address cannot be inferred from behavior alone.
- 6 The OpenArc evidence arc reveal.
- 7 Private local policies versus wallet-enforced policies.
- 8 What x402 adds to an investigation.
- 9 Why Gateway batching creates a separate settlement state.
- 10 The first Arc Testnet prototype walkthrough.
- 11 A deliberate incomplete-evidence demo.
- 12 What OpenArc will never collect.
- 13 Connector invitation for Arc and agent builders.
- 14 Testnet access or waitlist announcement after deployment.



SECTION 30

30. Social post bank

Thesis

A transaction hash tells you that value moved. It does not tell you who authorized an agent, which rule applied, what service was requested, or whether anything useful came back. OpenArc is being designed to connect that evidence.

Agent wallets solve execution. OpenArc is focused on the layer operators still need afterward: permission, action, payment, fulfillment, and settlement in one investigation.

The agentic economy does not only need faster payments. It needs better answers when someone asks: “Why did this happen?”

Product

See what was allowed. See what was attempted. See what was paid. See what settled. OpenArc brings the supplied evidence into one view without pretending it can see what no source recorded.

Authorized != settled. Settled != fulfilled. OpenArc is built around those differences.

A local policy can tell you what should have happened. Signed and onchain evidence can tell you what did. OpenArc is where the comparison becomes visible.

Privacy

Your agent names, operating limits, and investigation notes should not become another server-owned behavior graph. OpenArc is being designed around an encrypted local workspace.

OpenArc is not a wallet. It will not ask for a seed phrase, hold keys, or sign transactions. Oversight is the product.

Arc Testnet

We are starting with a narrow Arc Testnet question: can one interface reconcile an agent's policy, x402 payment, provider receipt, and final settlement without inventing the missing links?

OpenArc is an independent product being built first for Arc Testnet. It is not an Arc or Circle product. Arc has announced public mainnet for September 16, 2026; OpenArc mainnet support will not be claimed before the public production parameters and our own release gates exist.



SECTION 31

31. Thread scripts

Thread A: Why OpenArc exists

- 1 Software can now discover services and pay for them.
- 2 But a wallet history shows only part of the action.
- 3 The permission may live in one system.
- 4 The request may live in an agent trace.
- 5 The payment authorization may travel through HTTP.
- 6 The service receipt may be offchain.
- 7 Settlement may be batched later.
- 8 OpenArc is designed to connect those records without calling them the same kind of proof.
- 9 The goal: one human-readable evidence arc from permission to settlement.

Thread B: The six evidence classes

- 1 Local owner declaration.
- 2 Signed mandate or authorization.
- 3 Agent-reported event.
- 4 Provider receipt.
- 5 Facilitator record.
- 6 Onchain fact.
- 7 Each is authoritative for something different.
- 8 OpenArc preserves those boundaries and highlights conflicts.

Thread C: What OpenArc will not do

- 1 No custody.
- 2 No seed phrases.



- 3 No signing or broadcasting.
- 4 No claim to read hidden model reasoning.
- 5 No automatic agent identity from an address.
- 6 No broad “verified” label.
- 7 No mainnet claim before mainnet support is tested.
- 8 The product is evidence and investigation - not omniscience.



SECTION 32

32. FAQ and reply bank

What is OpenArc?

OpenArc is a proposed private evidence and investigation layer for economic agents. It is designed to connect supplied permissions, actions, payments, receipts, and settlement into one understandable timeline.

Is OpenArc a wallet?

No. OpenArc is designed to be read-only. It does not hold keys, collect seed phrases, sign transactions, or broadcast transactions.

Is OpenArc built by Arc or Circle?

No. OpenArc is an independent product concept. References to Arc, Circle, USDC, Gateway, Agent Stack, or x402 describe proposed interoperability and do not imply endorsement or partnership.

Is it available on Arc mainnet?

No. Arc has announced public mainnet for September 16, 2026, but OpenArc is an Arc Testnet build and no present-tense OpenArc mainnet claim is approved. Mainnet support requires official public production parameters, supported integrations, legal review, security review, and a separately verified release.

Why not call it a source of truth?

Different systems are authoritative for different facts. OpenArc can become a unified source-of-truth interface, but it cannot know private intent or offchain fulfillment unless that evidence is supplied.

Can it tell why an agent acted?

Only when a signed mandate, local declaration, or connected event supplies that context. A blockchain transaction alone cannot reveal private reasoning.

Can it stop an agent from spending?

The proposed MVP can compare actions with local monitoring policies. It should claim enforcement only when a connected wallet proves that it applied the rule.

What does it monitor?

The proposed first vertical covers one labeled Arc Testnet agent wallet, bounded USDC activity, an x402-style service payment, provider response evidence, and settlement state.

Does OpenArc read prompts?

Not by default. The product does not need raw prompts or hidden chain-of-thought to reconcile economic evidence. Optional connectors should emit bounded action events rather than private reasoning.

What is x402?

x402 is an HTTP payment-negotiation standard. A service can return payment requirements with a [402 Payment Required](#) response, and a client can retry with a payment authorization. The payment and settlement method remains a separate layer.

What is Gateway's role?

Circle Gateway can verify and batch eligible payment authorizations for later settlement. That means authorization, provider fulfillment, and final settlement may occur at different times and need separate states.

Does a provider receipt prove that the result was good?

No. It proves only the bounded provider claim that OpenArc received. Independent quality verification requires additional evidence.

How private is it?

The proposed design keeps labels, policies, notes, and evidence associations encrypted in a local workspace. It cannot protect an unlocked compromised device, malicious extensions, XSS, or information the user deliberately sends to a provider.

Will it support agents outside Circle Agent Stack?

That is the architectural goal, through a minimal connector schema. The first prototype should support only integrations that can be verified and clearly described.

Is there a token?

No token is defined by this package. Do not imply token utility, access rights, governance, investment value, or a launch plan.

SECTION 33

33. Content production guide

Recommended recurring formats:

- “One action, six evidence layers.”
- “What this transaction does not prove.”
- “Authorized / fulfilled / settled” comparison cards.
- “Missing evidence of the week.”
- Redacted Testnet investigation walkthrough.
- Policy-versus-action split screen.
- Connector spotlight describing exact fields and limitations.
- “What OpenArc refuses to infer” trust series.

Short-video formula:

- 1 Open with one investigation question.
- 2 Show the disconnected records for one second.
- 3 Trace the evidence arc.
- 4 Highlight the agreement or missing link.
- 5 Close with the product boundary: no custody, no signing, or no hidden-reasoning claim.

Visual requirements:

- Use only controlled Testnet transactions and fictional agent labels.
- Mark every staged example **TESTNET** or **SAMPLE**.
- Show source class and timestamp when a claim relies on them.
- Do not display real keys, raw payment signatures, private resource URLs, user prompts, or unredacted service responses.
- Do not use simulated metrics without an **ILLUSTRATIVE** label.



SECTION 34

34. CTA library

Concept phase:

- Follow the Testnet build.
- Read the evidence model.
- Tell us which agent-payment failure you need to investigate.
- Review the proposed connector schema.
- Join the design-partner list at [\[PRODUCT_URL\]](#).

Private alpha:

- Run one controlled Arc Testnet investigation.
- Connect a test wallet - not a production wallet.
- Review every source before marking the action reconciled.
- Export the redacted investigation report.

Public testnet after verification:

- Explore OpenArc on Arc Testnet.
- Start with one agent and one test payment.
- Compare the authorization with the final settlement.
- Inspect what OpenArc could - and could not - verify.



SECTION 35

35. Measurement

Do not use transaction volume, assets monitored, or customer counts until a real, auditable measurement exists.

Product-quality signals:

- Complete versus incomplete evidence arcs.
- Time to identify the failing evidence layer.
- Reconciliation conflicts detected.
- Late settlement updates resolved correctly.
- Connector availability and stale-evidence rate.
- Lock, encrypted backup, recovery, and deletion success.
- Percentage of explanations with visible supporting sources.

Adoption signals:

- Controlled investigations completed.
- Builders integrating the bounded event schema.
- Return use after an incomplete or conflicting action.
- Documentation completion and qualified design-partner requests.
- Support questions that expose missing states or confusing terminology.

Do not treat funds moved, token prices, or the number of autonomous actions as proof that OpenArc is useful or safe.



SECTION 36

36. Incident communication templates

Source outage

OpenArc cannot currently refresh [SOURCE]. Existing records remain visible with their prior observation time and may be stale. No new conclusion will be marked reconciled until the required source returns.

Conflicting evidence

OpenArc observed a mismatch between [EVIDENCE_A] and [EVIDENCE_B] for [ACTION_ID]. The action remains **CONFLICTING EVIDENCE**. Do not rely on the summarized outcome while the underlying amount, recipient, network, or status differs.

Delayed settlement

Payment authorization was observed, but final settlement has not yet been confirmed by the required source. The action remains **SETTLING**; it is not being reported as settled or failed.

Security issue

We have disabled [FEATURE OR CONNECTOR] while investigating a potential security issue. Do not connect new accounts or submit new evidence to that feature. Confirmed scope and next steps will be published at [STATUS_URL].

Incorrect reconciliation

OpenArc incorrectly reconciled [ACTION TYPE] under [AFFECTED VERSION]. We have disabled the affected conclusion and are rechecking impacted evidence states. The underlying wallet/provider/blockchain records remain authoritative.



SECTION 37

37. Publication checklist

- Capability is described as concept, Testnet, beta, or live according to its exact deployed state.
- Arc Testnet and Arc mainnet are not conflated.
- No Arc/Circle partnership or endorsement is implied.
- “Agent” classification has an identified source.
- “Verified” names the exact check.
- Authorization, fulfillment, and settlement remain distinct.
- Local policy is not called wallet-enforced without proof.
- No claim implies access to hidden reasoning.
- Privacy language names what stays local and the threat-model limits.
- Screenshots use fictional labels and controlled Testnet values.
- No private key, seed phrase, payment signature, account credential, private URL, or prompt appears.
- Product claims link to a deployed version or clearly say “designed”/“planned.”
- Mainnet dates and configuration come from current official sources.
- The CTA does not ask users to connect production funds to an unreviewed build.

SECTION 38

38. Source registry

Recheck these sources before any public launch because the Arc and agent-payment ecosystems are evolving rapidly.

Product sources:

- OpenArc technical specification: [docs/openarc-technical-spec.md](#)
- OpenArc product blueprint: [docs/openarc-product-blueprint.md](#)
- OpenArc brand system: [docs/openarc-brand-system.md](#)
- OpenArc launch plan: [docs/openarc-launch-plan.md](#)
- OpenArc roadmap: [docs/openarc-roadmap.md](#)

Current external references reviewed for this package:

- Arc mainnet announcement: <https://www.arc.io/blog/arc-mainnet-goes-live-on-september-16-2026>
- Arc network overview: <https://docs.arc.io/arc-chain>
- Arc deployment model: <https://docs.arc.io/arc/concepts/deployment-model>
- Arc RPC endpoints: <https://docs.arc.io/arc/references/rpc-endpoints>
- Arc EVM differences: <https://docs.arc.io/arc/references/evm-differences>
- Arc stablecoin-native model: <https://docs.arc.io/arc/concepts/stablecoin-native-model>
- Arc gas and fees: <https://docs.arc.io/arc/references/gas-and-fees>
- Arc contract addresses: <https://docs.arc.io/arc/references/contract-addresses>
- Arc opt-in privacy: <https://docs.arc.io/arc/concepts/opt-in-privacy>
- Arc post-quantum roadmap: <https://docs.arc.io/arc/concepts/post-quantum-security>
- Arc agentic-economy overview: <https://docs.arc.io/build/agentic-economy>
- Arc ERC-8004 quickstart: <https://docs.arc.io/arc/tutorials/register-your-first-ai-agent>
- Arc ERC-8183 quickstart: <https://docs.arc.io/arc/tutorials/create-your-first-erc-8183-job>
- Arc Testnet terms: <https://docs.arc.io/terms>
- Circle Agent Stack: <https://developers.circle.com/agent-stack>
- Circle Agent Stack overview: <https://www.circle.com/agent-stack>
- Circle Wallets supported chains: <https://developers.circle.com/wallets/supported-blockchains>
- Circle x402 and nanopayments concepts: <https://developers.circle.com/gateway/nanopayments/concepts/x402>
- Circle nanopayments: <https://developers.circle.com/gateway/nanopayments>
- Circle nanopayments SDK: <https://developers.circle.com/gateway/nanopayments/references/sdk>
- Circle Gateway supported chains: <https://developers.circle.com/gateway/references/supported-blockchains>
- Circle Gateway contracts: <https://developers.circle.com/gateway/references/contract-addresses>
- Circle x402 transfer search: <https://developers.circle.com/api-reference/gateway/all/search-x402transfers>
- Circle Agent Stack CLI: <https://developers.circle.com/agent-stack/circle-cli/command-reference>
- Circle Arc public Testnet announcement: <https://www.circle.com/pressroom/circle-launches-arc-public-testnet>
- Circle agentic-economy strategy: <https://www.circle.com/blog/building-the-financial-rails-for-the-agentic-economy>



- Google Agent Payments Protocol:
<https://cloud.google.com/blog/products/ai-machine-learning/announcing-agents-to-payments-ap2-protocol>
- x402 documentation: <https://docs.x402.org/introduction>

Ecosystem examples are useful evidence of demand but are not dependencies or proof of production maturity:

- NovaNet Arc session: <https://community.arc.network/public/clubs/agentic-economy-dofua/videos/trustless-usdc-agents-on-arc-2025-12-06>
- RSoft Agentic Bank session:
<https://community.arc.network/en/public/events/building-an-agentic-economy-on-arc-with-rsoft-agentic-bank-79o43gi2cn>

This sourcebook is product and communications planning, not legal, security, financial, or regulatory advice.